

PROBLEM STATEMENT 1

AI-Powered Email Attachment Threat Detection System

Background

Traditional antivirus systems such as ClamAV primarily rely on signature-based detection to identify known malware. While effective for previously identified threats, signature-based systems struggle with:

- Zero-day attacks
- Obfuscated malware
- Rapidly evolving phishing attachments
- Novel malicious document techniques

The objective of this hackathon is to design and build an ML-driven threat detection engine capable of identifying malicious email attachments and improving continuously through retraining and real-time feedback.

Objective

Build an AI-based system that:

1. Detects malicious email attachments (PDF, Office, ZIP, executables, etc.).
2. Identifies phishing-related documents and suspicious embedded content.
3. Scores attachments using ML instead of (or in addition to) signature matching.
4. Continuously retrains when new labeled datasets become available.
5. Learns incrementally from real-time customer feedback and sandbox outcomes.

Problem Scope

Participants must design a system that addresses the following:

1. File Classification

Given an uploaded email attachment:

- Extract relevant features.
- Predict whether the file is:
 - Benign
 - Malware
 - Suspicious
 - Phishing-related
- Provide a confidence score.

The system should handle common formats such as:

- PDF

- DOCX / XLSM (macro-enabled)
- ZIP archives
- Executables

2. Obfuscation & Anomaly Detection

The model should detect:

- High-entropy files (indicating encryption or packing)
- Suspicious macro usage
- Embedded JavaScript
- Nested archives
- Unusual structure patterns

3. Continuous Learning

The solution must support:

- Retraining when new labeled malware datasets are released (open-source feeds).
- Incorporating sandbox execution results into training data.
- Real-time learning or incremental updates based on:
 - User-reported false positives
 - Analyst-confirmed malicious samples

Technical Expectations

Participants should:

- Design an ML-based classification system.
- Justify model choice (e.g., XGBoost, CNN, Transformer, etc.).
- Implement:
 - Feature extraction pipeline
 - Model training pipeline
 - Inference API
- Provide retraining strategy documentation.
- Support real-time or near-real-time scoring.

Expected Deliverables

- Architecture diagram
- Trained model
- Inference API
- Retraining pipeline documentation

- Sample evaluation metrics
- Demo with test dataset

Evaluation Criteria

Submissions will be evaluated on:

- Detection accuracy (precision, recall, F1 score)
- False positive rate
- Model explainability
- Scalability and performance
- Retraining and continuous learning design
- Clean system architecture

Bonus Points

- Hybrid design combining:
 - Signature-based detection
 - ML-based scoring
- Sandbox behavioral analysis integration
- Model drift detection and automatic retraining triggers
- Explainability dashboard for security analysts

PROBLEM STATEMENT 2

AI for Preventive Air Quality Management & Sustainable Crop Residue Utilization

Background

Air pollution in major cities like Delhi is significantly impacted by seasonal agricultural residue burning in surrounding regions (e.g., Punjab and Haryana). Farmers often burn crop residue due to cost, time, and labor constraints, leading to severe air quality deterioration across large geographic areas.

This is not just a forecasting problem — it is a **prevention and intervention problem**.

The objective of this hackathon is to design an AI-driven system that:

1. Predicts large-scale biomass burning incidents before they occur.
2. Forecasts the downstream AQI impact on urban centers.
3. Recommends preventive actions.
4. Suggests economically viable alternatives to burning crop residue.
5. Designs incentive-aware, region-agnostic solutions applicable globally.

Objective

Build an intelligent system that combines:

- Satellite data
- Weather data
- Historical AQI records
- Agricultural cycle information
- Economic and logistics constraints

to predict, prevent, and economically redirect crop residue burning events.

Problem Scope

1. Burn Event Prediction (Preventive Model)

Given:

- Historical stubble burning data
- Satellite thermal hotspot data (e.g., MODIS / VIIRS)
- Weather conditions (wind speed, humidity, temperature)
- Crop cycle timelines
- Region-level agricultural activity data

Build a model that:

- Predicts probability of large-scale burn events 1–2 weeks in advance.

- Identifies high-risk districts.
- Estimates expected burn intensity.
- Flags early warning zones.
- Uses historical meteorological data

The model should generalize to any region globally where biomass burning affects air quality.

2. AQI Impact Forecasting

Build a dispersion-aware forecasting system that:

- Predicts AQI impact on downwind cities.
- Models wind transport of pollutants.
- Estimates severity and duration of pollution spikes.
- Quantifies expected health risk.

3. Preventive Intervention Recommendation Engine

Instead of just predicting burns, design a system that suggests:

- When and where intervention is required.
- Logistics support required (equipment, collection vehicles).
- Policy-based early-warning triggers.

The system should support:

- Government planning
- NGO coordination
- Regional mitigation strategies

4. Sustainable Waste Monetization Model

Design a recommendation model that:

- Suggests alternative uses of crop residue:
 - Organic fertilizer production
 - Biochar generation
 - Biomass energy conversion
 - Export as organic compost
- Estimates economic value per ton.
- Calculates cost vs benefit for farmers.
- Suggests optimal collection hubs and routes.
- Identifies buyers or markets.

The goal is to **transform waste burning into an income opportunity**, reducing the incentive to burn.

5. Continuous Learning & Global Applicability

The system must:

- Retrain when new satellite or AQI datasets are published.
- Learn from real-time sensor data.
- Adapt to different crops and regional climates.
- Be extensible to:
 - Wildfire-prone regions
 - Slash-and-burn agriculture zones
 - Biomass burning in Africa, South America, Southeast Asia

Technical Expectations

Participants should design:

- A predictive ML model for burn probability.
- A spatio-temporal model for pollution transport.
- An economic optimization model for waste conversion.
- A retraining pipeline triggered by new public datasets.
- A dashboard or API for intervention insights.

Possible model types:

- Spatio-temporal neural networks
- Gradient boosting for burn probability
- Graph Neural Networks for regional dispersion
- Reinforcement learning for intervention optimization
- Economic optimization algorithms

Evaluation Criteria

Solutions will be evaluated on:

- Burn prediction accuracy
- AQI forecast precision
- Intervention effectiveness simulation
- Economic feasibility modeling
- Scalability across regions
- Practicality of implementation

Expected Deliverables

- Architecture diagram
- Predictive models
- Intervention recommendation engine
- Retraining strategy
- Economic feasibility simulation
- Demo dashboard

Impact Vision

The goal is not just to measure pollution — it is to **prevent it at source while enabling economic upliftment**.

This solution should demonstrate how AI can:

- Improve air quality.
- Protect public health.
- Provide alternative income to farmers.
- Scale across regions globally facing biomass burning challenges.

PROBLEM STATEMENT 3

Intelligent Geospatial Engine with Real-Time Learning

Background

Modern businesses rely heavily on geospatial intelligence for routing, campus-level geofencing, address resolution, and live tracking. Most current systems depend on static routing engines and periodic map updates.

The objective of this hackathon is to design and build an ML-powered geospatial engine that:

- Supports intelligent routing
- Detects and manages geofences at global scale
- Performs geocoding and reverse geocoding
- Automatically retrains when new open-source map data is available
- Learns continuously from real-time customer telemetry

Objective

Develop a scalable ML system that can:

1. Optimize routes dynamically
2. Detect and classify geofence boundaries (e.g., global campuses)
3. Perform forward and reverse geocoding
4. Retrain automatically when new open-source map dumps are available
5. Continuously adapt to streaming real-time customer data

Use Cases

1. Intelligent Routing

Design a routing model that:

- Computes shortest or fastest paths
- Learns from historical traffic patterns
- Adapts to real-time telemetry
- Optimizes for cost, time, or configurable constraints

The model should improve as more travel data is collected.

2. Global Campus Geofence Identification

Given:

- Latitude and longitude streams
- Campus boundary datasets (complete or partially labeled)

Build a system that:

- Detects entry and exit events

- Identifies whether a coordinate belongs to a known campus
- Handles overlapping or ambiguous boundaries
- Supports dynamic geofence updates

Bonus: Automatically detect and cluster campus boundaries from movement data.

3. Geocoding and Reverse Geocoding

Build a model or hybrid system that:

- Converts address to coordinates
- Converts coordinates to structured address
- Handles incomplete or messy address inputs
- Learns from corrected user feedback

Bonus: Provide confidence scores and detect anomalies.

Continuous Learning Requirements

Open-Source Reinforcement

When new OpenStreetMap (or equivalent open-source data) dumps become available:

- Automatically trigger retraining
- Update routing graph features
- Recompute geofence models if necessary

Real-Time Learning

- Ingest live customer telemetry
- Update traffic weights, ETA predictions, or anomaly detection models
- Support incremental or streaming-based learning

Technical Expectations

Participants should:

- Use open-source map data (OpenStreetMap preferred)
- Build a reproducible training pipeline
- Design architecture that supports:
 - Offline training
 - Online inference
 - Streaming ingestion
- Justify model choices
- Provide evaluation metrics

Evaluation Criteria

Submissions will be evaluated on:

- Model accuracy (routing efficiency, ETA error, geofence precision)
- Scalability and architecture quality
- Real-time adaptability
- Retraining automation
- API design clarity
- Explainability and monitoring

Expected Deliverables

- Architecture diagram
- Trained model
- API endpoints for:
 - Routing
 - Geofence detection
 - Geocoding
- Retraining strategy documentation
- Working demo with sample dataset

PROBLEM STATEMENT 4

The Multi-Model Dynamic Feature Execution Challenge

Background

In large-scale ML platforms, **features are first-class citizens**.

A single request may trigger:

- Multiple ML models
- Each model requiring a different subset of features
- Significant overlap of features across models
- Complex inter-feature dependencies
- Strict latency and efficiency constraints

Feature computation is **expensive**, and recomputing the same feature multiple times is unacceptable.

Your task is to design a system that can **intelligently orchestrate feature execution** for multiple models in a single run.

The Challenge

You are given a complex feature dependency graph with the following characteristics:

Feature Dependency Graph (Hidden Complexity)

The dependency relationships between features are:

- Non-linear
- Multi-level
- Highly interconnected
- Fan-out and fan-in heavy

Some features:

- Are independent and can run immediately
- Unlock multiple downstream features
- Are required by several models simultaneously
- Sit deep in the dependency chain

The dependency graph:

- Is acyclic
- Is not shallow
- Is not guaranteed to be balanced
- Is intentionally designed to create multiple valid parallel execution paths

Participants must assume the dependency structure is non-trivial and production-grade.

Model Usage Pattern

- Any subset of models (M1–M4) may be requested per execution
- Models may:
 - Share multiple features
 - Have partially overlapping feature sets
 - Depend on features that are indirectly derived from others
- The system must determine:
 - Which features are needed
 - Which dependencies must be resolved
 - Which computations can be reused

⚠ Models are agnostic of dependencies.

They simply declare what they need, not how it should be computed.

Execution Characteristics

Your system must support:

- **Dynamic execution plans**
 - No predefined order
 - Determined at runtime
- **Maximum safe parallelism**

Expected Outcome

Design a system that can:

- Accept a request containing multiple models
- Infer the minimal required feature set
- Resolve a complex dependency graph
- Execute features efficiently and correctly
- Share computed features across models
- Complete execution deterministically and reliably

Core Constraint (Read Carefully)

Models do not orchestrate execution.

Features do not know about models.

The system must reconcile both dynamically.

What We're Evaluating

Participants will be evaluated on:

- Correctness of dependency handling
- Efficiency of parallel execution
- Feature reuse and avoidance of duplication
- Ability to scale as dependency complexity increases
- Clarity and robustness of system design

Bonus points for:

- Visualizing feature execution
- Clean abstraction between models, features, and orchestration
- Handling partial execution or feature failures
- Insightful metrics or observability

Your Mission

Build a Dynamic Feature Execution Engine capable of powering multiple models simultaneously—efficiently, correctly, and at scale—under complex and unpredictable dependency structures.